SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

SMART CLASSROOM OCCUPANCY AND RESOURCE MANAGEMENT SYSTEM

Mini Project — Computer Engineering

Sardar Patel Institute of Technology, Mumbai

Document detail	Value
Version / date	1.0	13 October 2026
Prepared by	Krish Shaw , Rayyan Shaikh , Ujjwal Singh
Project repository	github.com/rayyanm06/Smartclassroom

DOCUMENT CONTROL

Revision	Date	Description
1.0	13 Oct 2026	Initial project-specific SRS

TABLE OF CONTENTS

1. Introduction
2. Overall Description
3. Functional Requirements
4. External Interface Requirements
5. Use Case Specifications
6. System Architecture and Data Requirements
7. Non-Functional Requirements
8. Business Rules and Decision Logic
9. Verification and Acceptance Test Cases
10. Requirements Traceability
11. Risks, Limitations and Assumptions
12. Future Enhancements
Appendix A. API Endpoint Summary
Appendix B. Glossary
Appendix C. Repository Evidence and References

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification defines the scope, interfaces, constraints, functional behavior, and acceptance criteria of the Smart Classroom Occupancy and Resource Management System. It provides a common reference for the mini-project development team, guide, examiners, testers, and future maintainers. The specification has been prepared by examining the repository and documenting both existing implementation and intended operating requirements.

1.2 Problem Statement

Institutions often operate classrooms using fixed timetables and manually controlled lights and air conditioners without a consolidated view of actual room use. A scheduled class may be absent, an unscheduled room may be occupied, or appliances may remain active in an empty classroom. Separate sensor, camera and timetable information makes timely resource decisions difficult. This system aims to combine these evidence sources into explainable occupancy states and actionable facility alerts.

1.3 Objectives

- Maintain classroom metadata and time-based academic schedules.
- Accept occupancy evidence from a camera-based people detector and PIR/environment sensors (real or simulated).
- Classify rooms into understandable occupancy states, displaying confidence and reasons.
- Highlight timetable mismatches, energy wastage and device alerts for administrators and watchmen.
- Support AC/light commands through a backend command queue with device acknowledgements.
- Provide classroom history, campus analytics, utilization reports, CSV exports and simple occupancy estimates.

1.4 Scope and exclusions

In scope: a web interface with dashboard, classroom list/detail, camera console, timetable view, energy recommendations, analytics, predictions, alerts, reports, settings and a separate watchman view; a Python API with persistent relational storage; a configurable vision service; and a sensor simulator designed to use the same ingest contract as ESP32 devices.

Out of scope for the reviewed baseline: student identity recognition, facial recognition, individual attendance, fee management, assignment submission, LMS/course content, full physical electrical installation and guaranteed production authorization. A model registry table exists, but a trained, validated forecasting pipeline is not demonstrated in the current source tree.

1.5 Intended audience and definitions

Primary audience: project team, teachers/project guide, evaluators, QA testers, maintenance personnel. Terms: PIR = passive infrared motion sensor; DHT11 = digital temperature/humidity sensor; YOLO = object detection model; MJPEG = multipart JPEG video stream; telemetry = measurements reported by devices; occupancy fusion = combining evidence from timetable, camera and PIR; command queue = pending appliance actions collected by devices.

2. OVERALL DESCRIPTION

2.1 Product perspective

The system is a locally deployable multi-service application. A React single-page interface requests REST data from a FastAPI server. The API uses MySQL via SQLAlchemy and Alembic. A Python vision service reads a configured phone/RTSP/USB camera, runs YOLO person detection and posts counts. A separate Python simulator generates sensor readings and executes appliance commands in software. These services are independently runnable and exchange HTTP messages.

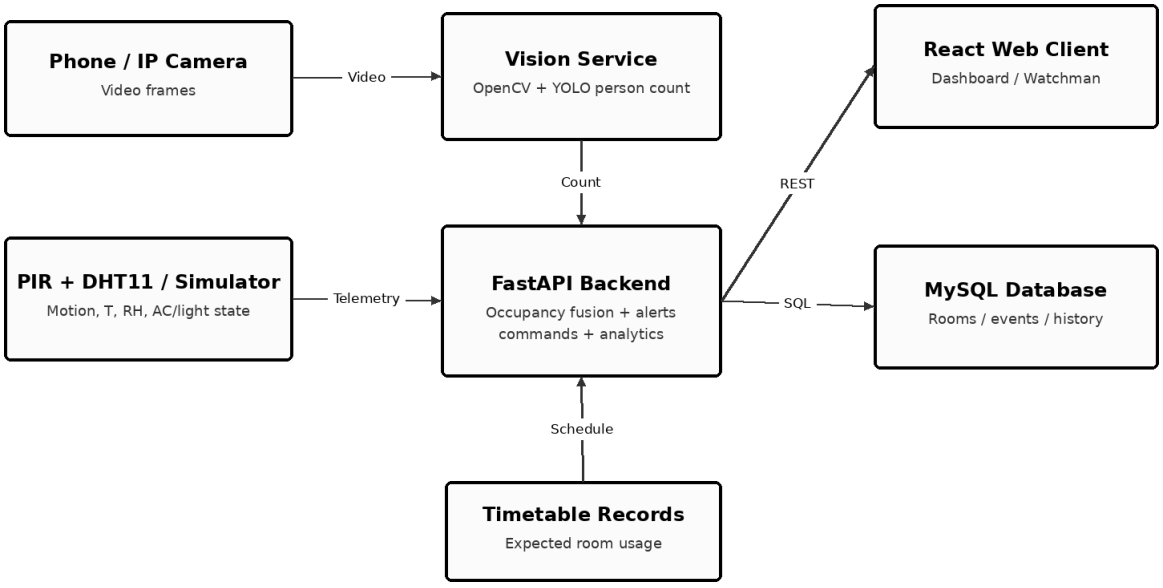


Figure 1. Deployment-level logical architecture (monochrome).

2.2 Operational modes

Mode	Behavior / boundary
Connected API mode	Frontend fetches FastAPI results; configured when VITE_USE_MOCKS=false. Backend, database and producers must be running.
Frontend mock mode	VITE_USE_MOCKS=true substitutes in-memory classroom, alert and chart data. This is demonstration data, not live occupancy.
Sensor simulation	sim/main.py posts explicitly labelled simulation readings for configured rooms and processes queued commands.
Physical camera	Vision reads configured source and posts person-count events; availability depends on network and camera connectivity.
Future ESP32 deployment	Actual PIR/DHT11/relay device can follow the documented sensor/command API; complete firmware is not present in the reviewed tree.

2.3 User classes

User class	Needs and privileges in current baseline
Administrative operator	Uses main web navigation to review room status, alerts, analytics, reports, settings and appliance actions. No account authentication is enforced in v1.
Watchman / facilities staff	Uses /watchman to read open alerts and acknowledge tasks. This is a separate UI route, not a server-enforced role.

Technician / developer	Configures device key, camera stream, simulation rooms, database, services and seeds. Uses API/docs and health endpoints.
Telemetry producer	Vision service / sensor simulator (or compatible device) posts validated sensor and camera payloads using X-Device-Key.

2.4 System constraints and dependencies

- Browser with JavaScript enabled and reachable frontend/API endpoints.
- Python 3.11+ is stated by the implementation specification; backend requirements include FastAPI, SQLAlchemy, Pydantic, PyMySQL and scikit-learn.
- MySQL is the configured persistence backend; API configuration expects DATABASE_URL.
- Vision depends on OpenCV, Ultralytics YOLO, suitable hardware/network feed and downloaded model weights.
- Timestamps should be handled consistently as UTC for storage and Asia/Kolkata for schedule interpretation.
- Room "508" is configured in the current camera YAML and frontend camera page; older planning text uses A101, therefore deployed IDs must be aligned to runtime configuration.

2.5 Feature availability baseline

Feature area	Current assessment	Evidence
Web dashboard, rooms, alerts, settings	Code	React pages + FastAPI routes
Sensor simulator + ingest	Code	sim/main.py, ingest.py
Camera detection and stream	Code; runtime dependent	vision/detector.py, vision/server.py
Timetable CRUD	API only; UI primarily read-only	routers/timetable.py; TimetablePage.tsx
Energy alerts and command queue	Code; actual switching depends on device	occupancy.py; command poll/ack
Analytics and CSV export	Code; some calculations use fallback/estimates	analytics.py; ReportsPage.tsx
Occupancy prediction	Rules/heuristics, not trained ML	analytics.py /predictions/occupancy
Admin login and server roles	Not implemented	No auth dependency on dashboard write routes
Automated production lifecycle / load test	Not established	Not evidenced by reviewed source

3. FUNCTIONAL REQUIREMENTS

Each requirement uses an unambiguous ID. "Shall" states expected behavior. Implementation status reflects static source inspection on the specified commit, not the results of executing the system.

3.1 Classroom inventory and dashboard

FR-01 — Campus dashboard. The system shall display total rooms, current occupancy breakdown, camera/sensor availability, selected environmental metrics and open energy alerts. *[Status: Code; Verification: Dashboard API + UI]*

FR-02 — Room listing. The system shall retrieve classrooms with IDs, locations, capacity, device capabilities and current occupancy state. *[Status: Code; Verification: GET /api/classrooms]*

FR-03 — Room details. For a selected classroom, the system shall display current/next timetable slots, live status, confidence, reasons and open alerts. *[Status: Code; Verification: GET /api/classrooms/{id}]*

FR-04 — Room maintenance. The API shall allow creation, updating and deletion of classroom records and initialize a default state for a newly created classroom. *[Status: API only; Verification: CRUD API tests]*

FR-05 — Historical room data. The system shall return time-ordered occupancy snapshots for a room with optional date filtering. *[Status: Code; Verification: History API]*

3.2 Telemetry, vision and occupancy

FR-06 — Sensor-event ingestion. The API shall accept PIR, environment and appliance telemetry for an existing room; reject absent/invalid device keys and unknown rooms. *[Status: Code; Verification: POST /api/sensor-data]*

FR-07 — Camera-event ingestion. The API shall accept camera people-count, confidence and timestamps for existing rooms and record detection events. *[Status: Code; Verification: POST /api/camera-data]*

FR-08 — Live camera view. The vision service shall provide an annotated MJPEG stream or explicit unavailable status for configured feeds. *[Status: Code, runtime dependent; Verification: Vision feed trial]*

FR-09 — Anonymous person detection. The optical pipeline shall count detected persons without identifying individuals or retaining raw frame recordings within the documented pipeline. *[Status: Code/design; Verification: Vision source review]*

FR-10 — Occupancy classification. The occupancy engine shall fuse camera count when available, recent PIR motion and expected timetable activity into a room state, confidence and textual reasons. *[Status: Code; Verification: Decision-table tests]*

FR-11 — Missing camera fallback. For rooms with no camera evidence, the engine shall use PIR and timetable; numeric people count may remain null/unknown. *[Status: Code; Verification: No-camera scenarios]*

FR-12 — Freshness and uncertainty. The application should distinguish stale camera/sensor evidence from a valid empty-room reading before operational action. Current code has freshness presets but does not consistently enforce them in the fusion path. *[Status: Partial; Verification: Stale telemetry tests]*

FR-13 — Occupancy snapshots. The system shall write periodic state snapshots to support historical analysis (the current evaluator checks roughly 60 seconds between snapshots). *[Status: Code; Verification: Database inspection]*

3.3 Timetable and scheduling

FR-14 — Timetable viewing. The web interface shall display the weekly schedule for a selected classroom. *[Status: Code; Verification: Timetable UI walkthrough]*

FR-15 — Timetable entry operations. The backend shall create, edit, list/filter and delete subject/faculty/time records. *[Status: API only; Verification: Timetable endpoint tests]*

FR-16 — Schedule clash detection. The API shall reject overlapping bookings in the same room and day with HTTP 409; adjacent end/start times shall not conflict. *[Status: Code; Verification: Overlap boundary tests]*

FR-17 — Expected occupancy window. The evaluator shall use day, local time, validity dates and a pre/post session buffer to determine expected usage. *[Status: Code; Verification: Clock-controlled tests]*

3.4 Energy and appliance workflow

FR-18 — Energy-waste detection. The engine shall flag an unoccupied room when its AC or lights are reported active and resolve alerts when conditions normalize. *[Status: Code; Verification: Energy alert scenario]*

FR-19 — Energy recommendation. The API shall list rooms with potentially wasted energy, rated-power assumptions and recommended off commands. Figures are estimates, not metered energy. *[Status: Code; Verification: Energy response check]*

FR-20 — Appliance command dispatch. A dashboard action shall create a pending AC/light command for an existing room; the UI must not treat pending as physically confirmed execution. *[Status: Code with caveat; Verification: Command lifecycle test]*

FR-21 — Device command receipt. An authenticated producer shall retrieve pending commands for its room, apply them where supported and acknowledge each command. *[Status: Code/simulator; Verification: Simulator integration]*

FR-22 — Command outcome integrity. Actual appliance status shall ultimately be reconciled with device telemetry, not only an optimistic API-side state update. This is a production-readiness requirement. *[Status: Partial; Verification: Compare command and sensor state]*

3.5 Alerts and facility operations

FR-23 — Alert generation. The system shall create/update occupancy anomaly and idle appliance alerts with room, severity, type, status and timestamps. *[Status: Code; Verification: Alert DB state]*

FR-24 — Alert triage. The UI and API shall list and filter alerts; a user shall be able to acknowledge or resolve an alert. *[Status: Code; Verification: Alert action APIs]*

FR-25 — Watchman task screen. A dedicated screen shall show current open facility alerts and support acknowledgement. *[Status: Code, view only role; Verification: /watchman interaction]*

3.6 Analytics, predictions and reporting

FR-26 — Occupancy analytics. The system shall expose expected-vs-observed occupancy curve, campus KPIs, room underutilization and floor-level environment summaries. *[Status: Code with estimation caveats; Verification: Analytics API]*

FR-27 — Occupancy estimate. The prediction screen shall accept classroom, weekday and hour and show estimated occupancy probability/headcount. Present implementation is timetable-rule heuristic; trained ML is future work. *[Status: Code (heuristic); Verification: Prediction endpoint]*

FR-28 — CSV exports. The web application shall allow downloading analytics and classroom audit data in CSV format. *[Status: Code; Verification: CSV content audit]*

FR-29 — Printable facility report. The web application shall show a printable audit report of classrooms and alerts. *[Status: Code; Verification: Print preview]*

3.7 Administration, deployment and safeguards

FR-30 — Configurable thresholds. The API shall store and retrieve settings, supporting named demo and production presets. *[Status: Code; Verification: Settings API]*

FR-31 — Device authentication. Device telemetry ingestion and device command collection/acknowledgement shall require X-Device-Key. *[Status: Code; Verification: 401 negative test]*

FR-32 — Administrative access control. Before deployment beyond a trusted demonstration network, administrative mutations and camera/report access shall use authenticated user accounts and role authorization. *[Status: Proposed; Verification: Security review]*

FR-33 — Demo data labeling. Simulated/seeded telemetry and derived charts should be visibly distinguishable from direct real measurements. Some code still uses mock and fallback estimates. *[Status: Partial; Verification: UI data provenance audit]*

FR-34 — Operational monitoring. The API and vision service shall expose health/status checks and indicate when dependent services are unreachable. *[Status: Partial; Verification: Failure injection]*

FR-35 — Retention. Telemetry retention shall be configurable and automatic for production; a raw-event purge helper is present but scheduled enforcement was not confirmed. *[Status: Partial; Verification: Retention job test]*

4. EXTERNAL INTERFACE REQUIREMENTS

4.1 Human interface

The system provides a desktop-oriented responsive web UI with routes / (dashboard), /classrooms, /classrooms/:id, /camera, /timetable, /energy, /analytics, /predictions, /alerts, /reports, /settings and /watchman. It also has an /intro landing sequence. The main navigation uses React Router and query-based polling, not a live WebSocket feed in the reviewed design.

Page	Principal content / action
Dashboard	Occupancy KPIs; room tiles; camera tile; charts; current alerts; acknowledge/resolve actions
Classrooms / detail	Filter/inspect rooms; reasons and confidence; local appliance controls; current/next class
Camera	Room 508 annotated feed and camera telemetry view (runtime camera source required)
Timetable	Weekly classroom timetable grid; no general timetable edit form observed
Energy	Idle appliance recommendations; queue AC/light off actions
Analytics / predictions	Utilization curve, KPIs, export; day/hour/room prediction console
Alerts / watchman	Browse alert stream; acknowledge actions
Reports / settings	Print or CSV export; choose and update threshold presets

4.2 Software interfaces and API conventions

Frontend calls the backend via HTTP/JSON at VITE_API_URL (default http://localhost:8000); video previews are provided by the separate vision service at VITE_VISION_URL (default http://localhost:8001). Most administrative API routes are currently unprotected. FastAPI exposes machine-readable OpenAPI docs at /docs. Error cases include 401 invalid device key, 404 missing room/alert and 409 timetable clash.

4.3 Hardware interface

Camera: configurable RTSP/MJPEG/USB input from a phone/IP webcam. Vision settings identify room ID, source URL, YOLO model, input size and detection confidence. Sensors: the simulator mimics PIR motion, DHT11 temperature/humidity and AC/light relay state; physical ESP32 firmware is an intended compatible producer rather than demonstrated complete integration. Physical device control requires electrical isolation, relay ratings and supervised deployment.

4.4 Communications

Telemetry transport: JSON POST to /api/sensor-data and /api/camera-data using X-Device-Key; commands: GET pending queue and POST acknowledgement; video: MJPEG HTTP stream. Frontend uses polling and ordinary REST calls. For production networks, HTTPS/TLS, restrictive CORS, secret rotation and identity-based operator access are required enhancements.

4.5 Key input validation boundaries

Field / action	Constraint evidenced in source
Sensor temperature	Nullable numeric, -10 to 60 °C
Sensor humidity	Nullable numeric, 0 to 100 %RH
Sensor source	simulation or esp32
Camera detected people	Non-negative integer
Camera confidence	0.0 to 1.0
Appliance action	device=ac|light; command=on|off
Timetable weekday	Integer 0..6 (Monday=0)

Timetable conflict	Existing same-room/day overlapping interval causes 409
Unknown classroom ingestion	404 response; system does not auto-create rooms

5. USE CASE SPECIFICATIONS

The use cases below describe operator-visible workflows, including API behavior. Actors are usage categories; operator authentication is not currently enforced.

UC-01 Monitor campus occupancy

Field	Specification
Primary actor	Administrative operator
Trigger	Opens dashboard
Preconditions	API and database available

Main success flow	1. Open the dashboard 2. Fetch summary, room states and alerts 3. Display occupied, empty, expected and anomaly information 4. Refresh from subsequent polling
Exceptions / alternative flow	Network/API failure must be shown rather than silently interpreted as zero occupancy.
Postcondition	Operator receives current contextual overview.

UC-02 Inspect a classroom

Field	Specification
Primary actor	Administrative operator
Trigger	Selects classroom
Preconditions	Room exists
Main success flow	1. Open room detail 2. Retrieve state, active/next timetable entries and open alerts 3. Inspect source reasons, climate values and appliance state
Exceptions / alternative flow	404 shows unavailable room; null counts must be presented as unknown.
Postcondition	Operator understands room state and evidence.

UC-03 Process sensor event

Field	Specification
Primary actor	Simulator / ESP32
Trigger	Sensor sample generated
Preconditions	Configured valid classroom and device key
Main success flow	1. POST sensor payload 2. Authenticate producer and validate schema 3. Persist reading and update room state 4. Run occupancy/alert evaluation and commit
Exceptions / alternative flow	401 bad key; 404 unprovisioned room; 422 invalid numeric range.
Postcondition	New sensor evidence is stored.

UC-04 Process live camera count

Field	Specification
Primary actor	Vision service
Trigger	Camera inference completes
Preconditions	Camera configured and stream available
Main success flow	1. Read frame 2. Run person-only YOLO detector 3. Send count/confidence to ingestion API 4. Update corresponding classroom evidence
Exceptions / alternative flow	If feed fails, live stream shows camera offline; detection cannot imply nobody is present.
Postcondition	Room count/status reflects available evidence.

UC-05 Manage timetable via API

Field	Specification
Primary actor	Technician / administrator (API)
Trigger	Add or edit schedule
Preconditions	Existing classroom

Main success flow	1. Submit room/day/start/end/subject/faculty 2. Validate fields and check overlap 3. Persist change 4. Display updated schedule in read view
Exceptions / alternative flow	Same-room overlapping slots receive 409; missing room receives 404.
Postcondition	Schedule is valid and available for occupancy fusion.

UC-06 Handle energy-waste warning

Field	Specification
Primary actor	Administrative operator
Trigger	Empty room reported with appliance on
Preconditions	Occupancy engine running
Main success flow	1. Generate idle AC/light alert 2. Show estimated waste and recommendation 3. Operator requests off action 4. Create pending command
Exceptions / alternative flow	False occupancy or stale data may cause false positives; requires human confirmation.
Postcondition	Action queued for device; not yet guaranteed applied.

UC-07 Apply command in simulator

Field	Specification
Primary actor	Sensor simulator
Trigger	Pending queue polled
Preconditions	Valid device key and corresponding simulated room
Main success flow	1. GET pending commands 2. Apply on/off to software appliance 3. POST command acknowledgement 4. Transmit subsequent telemetry
Exceptions / alternative flow	If simulator disconnected, command remains pending; do not report completed as hardware success.
Postcondition	Command status moves to acked after simulated application.

UC-08 Acknowledge an alert

Field	Specification
Primary actor	Watchman
Trigger	Opens /watchman
Preconditions	Open alert exists
Main success flow	1. Fetch open alert list 2. Review location and problem 3. Select acknowledge 4. Update alert status on API
Exceptions / alternative flow	Concurrent resolution or missing ID returns 404.
Postcondition	Alert is acknowledged with timestamp.

UC-09 Export compliance report

Field	Specification
Primary actor	Administrative operator
Trigger	Click export CSV / print
Preconditions	Classroom and alert data loaded
Main success flow	1. Fetch room/alert data 2. Format current table and summary 3. Download CSV or open browser print dialog
Exceptions / alternative flow	Report correctness depends on source freshness and any estimates.

Postcondition	Local CSV/print artifact available.

UC-10 Predict room use

Field	Specification
Primary actor	Administrative operator
Trigger	Select room/day/hour
Preconditions	Room exists in DB
Main success flow	1. Request prediction endpoint 2. Check scheduled entry 3. Return rule-derived probability and headcount estimate 4. Display factors
Exceptions / alternative flow	Unknown room returns an error object; probabilities are not measured model confidence.
Postcondition	An explicitly estimated future occupancy result appears.

UC-11 Select threshold preset

Field	Specification
Primary actor	Administrative operator
Trigger	Change demo/production preset
Preconditions	Backend settings accessible
Main success flow	1. Request preset switch 2. Persist preset values in database 3. Refresh settings screen
Exceptions / alternative flow	Unknown preset receives 400; some evaluator constants do not yet use saved thresholds.
Postcondition	Preset record changes.

6. SYSTEM ARCHITECTURE AND DATA REQUIREMENTS

6.1 Data processing flow

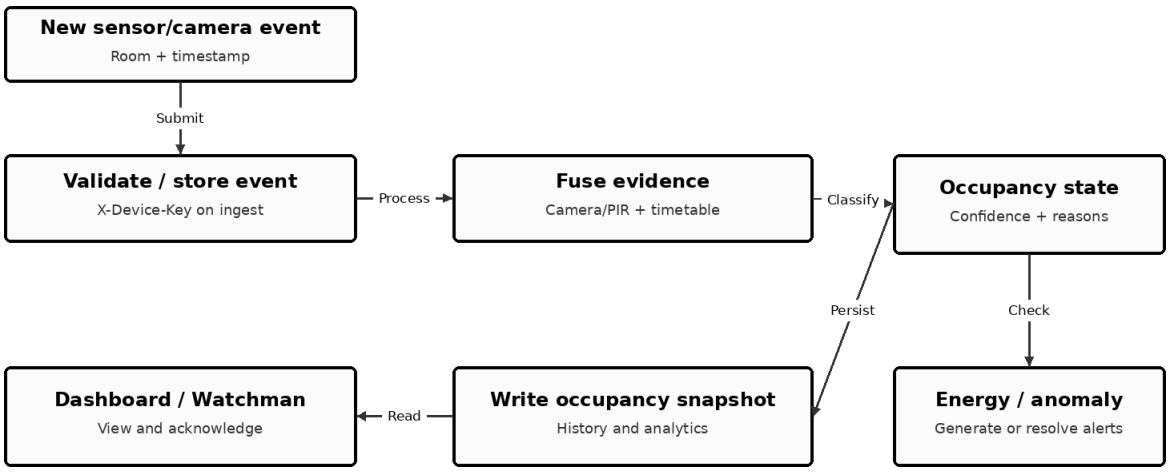


Figure 2. Conceptual event-to-decision data flow. Some runtime stages execute on API reads rather than a dedicated background worker.

6.2 Major components

Component	Responsibility	Code location
Web client	Routing, dashboard, charts, form selections, alert actions, exports	apps/web/src/
REST API	Ingestion, classroom operations, timetable, analytics, settings and command lifecycle	services/api/app/
Occupancy engine	Classify state and raise/resolve energy and mismatch alerts	services/api/app/engine/occupancy.py
Vision	Read live image source; YOLO person counting; annotated MJPEG	services/vision/vision/
Simulator	Generate PIR/DHT11 events; command application & ack	services/sensor-sim/sim/
Database	Persistent rooms, telemetry, snapshots, alerts, predictions metadata	services/api/app/models/

6.3 Principal database entities

Entity / table	Key fields and associations
classrooms	PK id; building/floor/capacity; has_camera/PIR/DHT; rated AC/light powers; active
classroom_state	PK/FK classroom_id; occupancy_state; confidence; reasons JSON; count; latest sensor/camera/energy fields
timetable_entries	PK id, FK classroom_id; subject, faculty, weekday, start/end, class type, validity range
sensor_readings	PK id; classroom_id, device_id, PIR, T/RH, AC/light, source, timestamps
camera_detections	PK id; classroom_id, camera_id, people_detected, confidence, source, timestamps
occupancy_snapshots	PK id; classroom_id, ts, state, count, expected, temperature, appliance states, source
alerts	PK id; classroom_id, type, severity, status, headline/detail, seen/ack/resolved times
device_commands	PK id; classroom_id, device, command, status, request/ack time
settings	PK id; unique key; JSON value; preset; updated_at
prediction_models	PK id; algorithm, metrics, feature_importance, artifact path (schema exists; active training not evidenced)

6.4 Conceptual entity relationships

A Classroom has one ClassroomState; one Classroom may have many TimetableEntries. Room IDs also associate sensor readings, camera detections, occupancy snapshots, alerts and device commands with the classroom. The current historical/alert models store classroom_id rather than always defining a database-level foreign key; referential integrity for those records requires application validation and integrity tests.

Cardinality summary: CLASSROOM (1) -> (0..1) CLASSROOM_STATE; CLASSROOM (1) -> (0..N) TIMETABLE_ENTRY; CLASSROOM (1) -> (0..N) SENSOR_READING, CAMERA_DETECTION, OCCUPANCY_SNAPSHOT, ALERT and DEVICE_COMMAND.

6.5 Data integrity, accuracy and provenance

Sensor readings declare source as simulation or esp32; snapshots declare live or seed. Camera detections store count/confidence metadata, not images. Operational dashboards must avoid presenting mock data, rule-based

estimates or default sensor fallbacks as authenticated physical measurements. Since camera and PIR measurements may be unavailable, the UI needs null/unknown states and explicit health indicators.

6.6 Data lifecycle

On accepted input, store the raw event, update the current room state and evaluate alerts. Occupancy snapshots are written no more often than the evaluator's approximate one-minute limit. A helper can delete sensor and camera raw events older than seven days; the reviewed source does not establish a scheduled recurring purge job. Retention of snapshots, exports, backups and access logs should be defined before production deployment.

7. NON-FUNCTIONAL REQUIREMENTS

The following measurable values are proposed acceptance targets unless stated as an implemented mechanism. They were not established by benchmark runs on the reviewed repository.

ID	Quality attribute	Measurable requirement / target	Status
NFR-01	Usability	Primary dashboard should load clearly on a 1366x768 desktop and maintain readable status labels on tablet-width browsers.	Proposed
NFR-02	Response time	Under a local pilot load (up to 25 rooms / 5 viewers), 95% of ordinary API reads should return within 2 seconds.	Proposed
NFR-03	Freshness	Show last-seen age. Detect outdated sensors (>30 s) and camera events (>15 s) via configurable thresholds before claiming high-confidence occupancy.	Partially designed
NFR-04	Availability	Disconnected camera/simulator must not crash API or entire dashboard; show degraded status.	Partially coded
NFR-05	Privacy	No facial identity, biometric enrollment or stored raw classroom images in normal operation.	Design/code
NFR-06	Security	HTTPS, per-device credentials, role enforcement for operator mutation routes and restrictive CORS before campus deployment.	Proposed; gaps present
NFR-07	Integrity	Unauthorized device ingestion returns 401; unknown classroom returns 404; conflicting timetable slot returns 409.	Code
NFR-08	Maintainability	Separate UI, API, vision and simulator components; document environment vars and runbooks; automated tests for fusion and command lifecycle.	Mixed

NFR-09	Observability	Expose API and vision health, camera/source status and command pending/ack state; log failures safely.	Partial
NFR-10	Reliability	Lost telemetry or missing database connectivity shall produce explicit degraded/unknown states, not fabricated live statistics.	Proposed
NFR-11	Accessibility	Controls shall be keyboard reachable; occupancy meaning shall not rely only on colors; text contrast shall remain readable when printed B/W.	Proposed
NFR-12	Interoperability	Use versioned JSON API and HTTP to allow simulator-to-ESP32 transition without changing the frontend.	Code/design
NFR-13	Backup/recovery	Define DB backups and restore trial before field use; target maximum data loss <= 24 h for historical metadata.	Proposed
NFR-14	Camera latency	Campus pilot target < 1 second live visual latency where network/hardware allow; requires real measurement.	Proposed

8. BUSINESS RULES AND DECISION LOGIC

8.1 State definitions

State	Meaning in reviewed occupancy logic
OCCUPIED	Camera persons >0 or (without usable camera count) recent PIR motion, with class expected
EMPTY	Zero detected persons or no recent PIR evidence, when no class expected
EXPECTED_OCCUPANCY	Timetable expects class, but no presence yet and PIR-only idle below current hard-coded grace window
UNEXPECTED_OCCUPANCY	Physical presence indicated outside an expected timetable window
OCCUPANCY_ANOMALY	Class expected but camera count=0, or PIR-only no-motion persists beyond hard-coded grace window
SENSOR_UNCERTAIN	Represented in schema/UI counters but not actively produced by the current inspected decision branch

8.2 Decision table

Camera evidence	Recent PIR	Expected schedule	Decision / confidence
Person count > 0	Any	Yes	OCCUPIED / high
Person count > 0	Any	No	UNEXPECTED_OCCUPANCY / high
Person count = 0	Any	Yes	OCCUPANCY_ANOMALY / high
Person count = 0	Any	No	EMPTY / high
None / not usable	Yes	Yes	OCCUPIED / medium

None / not usable	Yes	No	UNEXPECTED_OCCUPANCY / medium
None / not usable	No	Yes; short idle	EXPECTED_OCCUPANCY / low
None / not usable	No	Yes; idle >= 5 min	OCCUPANCY_ANOMALY / medium
None / not usable	No	No	EMPTY / high

8.3 Implementation-specific observations

- Current engine treats PIR motion within approximately 180 seconds as recent and uses a 5-minute no-motion anomaly threshold, despite separate stored demo/production settings.
- Schedule expectation uses an implementation call with pre-start 5 minutes and post-end 10 minutes.
- Energy alerts are generated when occupancy state is EMPTY or OCCUPANCY_ANOMALY and the appliance state is ON; current evaluated rule does not apply the configurable idle threshold before every alert.
- If an appliance action is queued, the classroom state can be updated immediately on the API before a device acknowledgement; real physical completion must be verified separately.
- Some analytics paths use hard-coded baseline totals/default temperatures or inferred occupancy points when history is missing; estimated values must not be interpreted as historical sensor measurements.
- Probability values (e.g., higher for a scheduled lecture and lower for a free slot) come from rules in the prediction endpoint, not a documented trained forecasting model.

8.4 Energy estimation rule

For a flagged empty room, estimated waste (kWh) = [AC rated power (kW) if ON + lighting rated power (kW) if ON] x idle time (hours). The current endpoint uses at least one minute for the calculation. This gives a rough opportunity estimate; without metering or a validated runtime window, it is not an actual billable energy saving.

9. VERIFICATION AND ACCEPTANCE TEST CASES

Execution status: these are planned functional acceptance cases; this SRS does not claim the tests have been run successfully in a live installation. Existing pytest files cover selected classroom and sensor scenarios, but comprehensive deployment validation remains a separate activity.

Test	Req.	Input / scenario	Expected result
TC-01	FR-02/04	Create unique classroom, list, update and delete	201/create; listed; changes reflected; 204/delete
TC-02	FR-04	Create duplicate room ID	409 conflict
TC-03	FR-06/31	POST sensor-data with wrong/missing device key	401 unauthorized; no event accepted
TC-04	FR-06	POST temperature outside -10..60	422 validation error
TC-05	FR-06	POST event for unknown classroom	404 not found
TC-06	FR-07	POST camera count=4 / confidence=.9 for known camera room	202; detection recorded; state updated
TC-07	FR-10	Positive camera count + scheduled class	OCCUPIED with high confidence
TC-08	FR-10	Positive camera count + no class	UNEXPECTED_OCCUPANCY
TC-09	FR-10	Zero camera count + scheduled class	OCCUPANCY_ANOMALY and warning
TC-10	FR-11	No camera, recent PIR + scheduled class	OCCUPIED with medium confidence

TC-11	FR-12	Unplug camera and wait past freshness threshold	Expected improvement: show unavailable/uncertain, not stale high-confidence count
TC-12	FR-15/16	Create 09:00-10:00, then 09:30-10:30 same room/day	Second request 409
TC-13	FR-16	Create 10:00-11:00 adjacent to 09:00-10:00	Allowed; no overlap
TC-14	FR-18	Empty room reports AC ON	Open ENERGY_AC_IDLE alert; recommendation visible
TC-15	FR-20/21	Issue AC OFF; simulator polls and ACKs	Pending then acked; later telemetry matches OFF
TC-16	FR-23/24	Acknowledge open alert, then resolve	status and timestamps updated
TC-17	FR-25	Open /watchman with open alerts	Tasks visible and acknowledge works
TC-18	FR-26	Open analytics with absent snapshots	Estimates visibly labelled; no deceptive live claims
TC-19	FR-27	Predict scheduled/free hour	Different rule-based probabilities and relevant schedule metadata
TC-20	FR-28/29	Export reports CSV and browser print	Valid headers/rows; print fits page
TC-21	FR-30	Switch demo -> production setting	Correct preset and persisted values
TC-22	FR-32	Attempt unauthenticated appliance mutation	Target behavior after auth work: reject; baseline lacks this control
TC-23	NFR-04/09	Stop vision service while API remains on	Explicit camera unavailable; no full-system crash
TC-24	NFR-05	Inspect persistence after a camera cycle	No raw video frames or face identifiers stored

10. REQUIREMENTS TRACEABILITY

The table connects business goals to concrete requirements, code areas and representative tests. “Evidence” describes what to inspect rather than asserting that a runtime validation has passed.

Goal	Requirement IDs	Evidence / main module	Acceptance tests
Room visibility	FR-01..05	web dashboard/classroom pages; routers/classrooms.py	TC-01, TC-02
Reliable telemetry	FR-06..09, FR-31	routers/ingest.py; vision; sensor-sim	TC-03..06, TC-24
Occupancy reasoning	FR-10..13, FR-17	engine/occupancy.py; engine/schedule.py	TC-07..11
Scheduling	FR-14..16	routers/timetable.py; TimetablePage.tsx	TC-12, TC-13
Energy control	FR-18..22	dashboard.py; classrooms.py; sim/main.py	TC-14, TC-15
Alert operations	FR-23..25	alerts.py; WatchmanPage.tsx	TC-16, TC-17
Decision support	FR-26..29	analytics.py; PredictionsPage; ReportsPage	TC-18..20
Configuration/safety	FR-30..35	settings.py; security.py; retention.py	TC-21..24

11. RISKS, LIMITATIONS AND ASSUMPTIONS

11.1 Key risks and mitigations

Risk / current limitation	Effect	Required mitigation / owner
No authenticated admin/watchman account; open CORS	Unauthorized users on reachable network could mutate settings/commands or read camera information	Add user login, RBAC, CORS allowlist; deployment owner
Example keys and database credentials in configuration defaults	Credential reuse/exposure risk	Use secret manager/environment; rotate sample keys and passwords
Camera room ID discrepancy in old spec vs current config	Wrong-room stream or count association	Use current configured 508; reconcile deployment mapping
Freshness values saved but not consistently enforced by evaluator	Stale evidence may lead to incorrect occupancy and control decisions	Implement stale-data state and sensor heartbeat checks
Simulator / mock / fallback chart data	Numbers may be mistaken for production evidence	Show provenance badges/labels; disable misleading fallbacks
Optimistic appliance state update at command creation	False implication that AC/light was physically switched	Reconcile using acknowledged command plus measured state
Heuristic “prediction” labelled ML in some UI text	Overstates intelligence and unvalidated accuracy	Label rules as estimates; add/train/validate ML before claim
No demonstrated deployed ESP32/relay firmware	Only simulated action lifecycle fully reviewable	Document physical commissioning tests and electrical safety
Manual camera placement and Wi-Fi isolation	Intermittent video/connectivity; false zero counts	Dedicated network, device health, clear offline treatment
Snapshot/analytics estimates and sparse historical data	Misleading aggregated reports	Historical coverage checks; separate observed from computed

11.2 Assumptions

- Each classroom has a unique stable ID, and a timetable record refers to a provisioned classroom.
- The installation has consent and authority to operate an anonymous, non-identifying camera in classroom spaces.
- Sensor simulator readings are for development and demonstration; actual electricity use is not measured by DHT11/PIR.
- A real camera or simulator may be unavailable during demonstrations, so the UI must signal source quality.
- The main SRS is based on static code and documentation inspection, not physical acceptance testing.

11.3 Known scope gaps

No end-user authentication/authorization, no evidence of fully working ESP32 hardware control, no trained predictive model, no proven production-grade stale-data handling, and no end-to-end benchmark results were available in the inspected code. These are explicitly not claimed as completed capabilities.

12. FUTURE ENHANCEMENTS

- Add login with administrator, facility staff and view-only roles and audit logs for state-changing actions.
- Replace simulator with tested ESP32/PIR/DHT11/relay hardware using per-device keys and TLS.
- Create an end-to-end device heartbeat, stale-input state and device-command lifecycle with acknowledgement verification and timeout handling.
- Train and evaluate an interpretable occupancy prediction model from quality-controlled snapshots; report error metrics and data provenance.
- Implement scheduled retention/backup policies, production monitoring and calibrated camera-count accuracy tests.
- Add in-UI timetable editors, room provisioning forms and richer filtering and safe notification workflows.

APPENDIX A. API ENDPOINT SUMMARY

The table lists central routes inspected in source. The OpenAPI file and FastAPI /docs are the authoritative contracts for detailed payload schemas. No login should be presumed for read/write routes other than the explicitly protected device operations.

Method	Path	Purpose
GET	/api/health	API/DB health
GET	/api/dashboard/summary	Campus KPI summary
GET	/api/dashboard/energy-recommendations	Estimated idle-appliance waste
GET/POST	/api/classrooms	List/create classrooms
GET/PUT/DELETE	/api/classrooms/{id}	Detail/update/delete room
GET	/api/classrooms/{id}/history	Occupancy snapshots
POST	/api/classrooms/{id}/actions	Queue AC/light command
POST	/api/classrooms/{id}/camera-telemetry	Additional unprotected telemetry update route
GET/POST	/api/timetable	List/create timetable
PUT/DELETE	/api/timetable/{id}	Change/remove timetable entry
POST	/api/sensor-data	Protected sensor ingest
POST	/api/camera-data	Protected camera event ingest
GET	/api/classrooms/{id}/commands/pending	Protected device command fetch
POST	/api/commands/{id}/ack	Protected command ack
GET	/api/alerts	List alerts
POST	/api/alerts/{id}/acknowledge	Acknowledge
POST	/api/alerts/{id}/resolve	Resolve
GET/PUT	/api/settings	Retrieve/update thresholds
POST	/api/settings/preset/{name}	Switch preset
GET	/api/analytics/utilization	Hourly utilization trend
GET	/api/analytics/kpis	Campus KPIs
GET	/api/analytics/underutilized	Low-utilization rooms
GET	/api/analytics/environment	Temperature by floor
GET	/api/predictions/occupancy	Rule-based probability
GET	vision /health, /status	Vision health/metadata
GET	vision /stream/{id}.mjpg	Annotated MJPEG stream

APPENDIX B. GLOSSARY

Term	Meaning
ESP32	Microcontroller platform intended for real sensor input and relay output
PIR	Motion sensing without counting identifiable people
DHT11	Temperature/humidity sensor (limited precision)
YOLO11n	Ultralytics person-detection object model in current camera config
MJPEG	HTTP streaming of successive JPEG images
Occupancy anomaly	Scheduled class indicated but physical evidence suggests room unoccupied
Telemetry	Sensor/camera measurement transferred to API
Mock adapter	Frontend local sample data chosen by VITE_USE_MOCKS
Device key	Shared secret header used by device-facing API routes
Snapshot	Persisted point-in-time occupancy reading

APPENDIX C. REPOSITORY EVIDENCE AND REFERENCES

Reviewed source: https://github.com/rayyanm06/Smartclassroom (branch: main; commit: 1563c766b73b8737d236cd16fe2501934daefc5d; published 5 October 2026). Source review date: 13 October 2026. The document distinguishes original planning assertions from observed code; it does not represent a formal runtime audit.

- Project intent/planning: docs/SPEC.md and SMART_CLASSROOM_SPEC.md; decisions: docs/DECISIONS.md.
- Frontend views and integrations: apps/web/src/app/router.tsx; pages/*.tsx; services/api.ts; services/mocks/data.ts.
- Backend routes and core rules: services/api/app/routers/*.py; engine/occupancy.py; engine/schedule.py; core/security.py and settings_defaults.py.
- Data definitions: services/api/app/models/*.py; schemas/*.py; Alembic initial migration.
- Camera pipeline: services/vision/vision/{capture,detector,server,main}.py; config/cameras.yaml.
- Sensor simulation: services/sensor-sim/sim/*.py; config/rooms.yaml.
- Existing automated tests: services/api/app/tests/test_classroom_crud.py, test_sensor_ingest.py; services/sensor-sim/tests/test_simulation.py.

Final note on verification

This is an SRS based on the available repository, not a statement that all features have been executed on the intended network, physical sensors or database. For academic submission, the project team should run the acceptance cases and attach evidence of successful tests; implementation gaps are deliberately disclosed here to preserve accuracy.